

Design a Digital Wallet Payment System

The senior engineer answer sheet

One person sends money to another. Everything hard is underneath.

A customer sends one hundred. The money leaves their wallet. The service crashes. The receiver never gets it.

That is not mainly a database problem. It is a system design problem, and it is the one this sheet answers.

Every technical statement below was checked against the vendor documentation named beside it. Where something is a general principle rather than a documented behaviour, it says so.

In scope

- Check a wallet balance
- Transfer value between two wallets on the same platform
- Transfer status and history
- Prevent double spending
- Survive retries
- Keep an auditable record
- Notify both parties after success

Deliberately out of scope

- Merchant payments, which bring authorisation versus capture, acquiring, refunds, fees and chargebacks
- Agent cash in and cash out
- KYC and sanctions screening inside the transaction path
- Foreign exchange and cross border settlement
- Card and bank rails, custody, airtime, lending

Narrowing the scope out loud is itself a senior move. A candidate who designs everything designs nothing.

00 Scale assumptions

Registered wallets

10 million

Daily active users	1 million
Transfers per day	3 million
Average rate	about 35 per second
Designed peak	about 1,000 per second

These are interview assumptions chosen to make the numbers concrete. They are not published figures from any provider, and no claim is made here about how any named company works.

01 The answer most candidates give, and why it fails

Read the sender balance. Subtract in application code. Write the new number back. Then do the same in reverse for the receiver.

Two things kill it, and a senior interviewer is waiting for both.

PostgreSQL runs at Read Committed by default, so a SELECT sees only data committed before that statement began, and two SELECTs in one transaction can see different data.

PostgreSQL 18, Transaction Isolation

Read Committed prevents dirty reads, but permits nonrepeatable reads, phantom reads and serialization anomalies.

PostgreSQL 18, Transaction Isolation

Two transfers that read the same starting balance and each write an absolute result will silently overwrite one another. This is what the literature calls a lost update. The PostgreSQL manual does not use that phrase, so do not attribute it to them.

Standard database literature, not the PostgreSQL manual

Debiting the sender and crediting the receiver as two separate committed steps creates a window where the money exists nowhere. A crash inside that window is not recoverable by retrying, because the first step already committed.

First principles, not vendor documentation

02 Ledger, not a mutable balance

Stop storing the balance as a number you edit. Store what happened, as entries that never change, and derive the balance from them.

Double entry means every movement is written as a matched pair: one account debited, another credited, in one journal entry, summing to zero. The invariant is not "the balance looks right", it is "the entries balance".

A mutable balance can be wrong with no evidence. A ledger cannot be wrong quietly: if it is wrong, the entries do not sum, and that is detectable.

A single transaction commits all of its row changes or none of them, so both sides of the entry land together or neither does.

PostgreSQL 18, Transaction Isolation

Holding a reserved or pending balance separately from the posted balance earns its place only when completion depends on something that cannot join the transaction: a card authorisation, a bank payout, an external settlement provider. For two wallets inside one consistency boundary it is ceremony, because the transaction already solves it.

First principles, not vendor documentation

03 Two transfers at once

Correctness under concurrency is a choice you make explicitly, and the interviewer wants to hear which one you chose and why.

SELECT FOR UPDATE locks the rows it returns so they cannot be locked, modified or deleted by other transactions until the current transaction ends.

PostgreSQL 18, Explicit Locking

Row level locks do not affect ordinary reads. They block only writers and lockers to the same row, so a plain SELECT still sees the row.

PostgreSQL 18, Explicit Locking

At Read Committed, SELECT FOR UPDATE waits for a competing transaction and then locks and returns the updated row.

PostgreSQL 18, Explicit Locking

At Repeatable Read or Serializable it does not wait. An error is thrown if the row to be locked has changed since the transaction started. The same statement, two behaviours, decided by isolation level.

PostgreSQL 18, Explicit Locking

Repeatable Read and Serializable transactions can fail with a serialization error, and the documented remedy is to abort and retry the whole transaction from the beginning. If you choose those levels, retry is part of your design, not an afterthought.

PostgreSQL 18, Transaction Isolation

PostgreSQL Repeatable Read is stronger than the SQL standard requires. The standard permits phantom reads at that level; PostgreSQL does not allow them.

PostgreSQL 18, Transaction Isolation

04 The retry that must not pay twice

A client times out and retries. The first request may have succeeded. The network cannot tell you which.

The fix is an idempotency key supplied by the caller, stored with the result of the first attempt. The second arrival returns the stored result rather than moving money again.

The key is scoped to the operation, not to the user, and it must be persisted in the same transaction that performs the transfer. Stored afterwards, a crash between the two leaves the money moved and the key missing, which is the exact bug it was meant to prevent.

First principles, not vendor documentation

The same key arriving with a different request body is a client error, not a duplicate. Returning the original result would hide a real bug, so reject it.

First principles, not vendor documentation

05 Telling the rest of the world, without lying

The transfer committed. Now a notification must go out, and reporting and risk want to know.

Writing to the database and publishing to a broker are two systems. They cannot commit together, so one of them can succeed alone. Publish first and you can announce money that never moved. Publish after and you can move money nobody hears about.

The outbox pattern makes the message part of the same transaction: the event is inserted into an outbox table alongside the ledger entries, and a separate publisher reads that table and sends it.

The outbox makes the decision to send atomic with the money movement. It does not make the send itself exactly once: the publisher can crash after sending and before marking the row, so the same event can go out twice. Consumers must still be idempotent.

First principles, not vendor documentation

Kafka names three delivery semantics. At most once, messages may be lost but are never redelivered. At least once, messages are never lost but may be redelivered. Exactly once, each message is processed once and only once.

Kafka 4.3, Design: Message Delivery Semantics

The Kafka producer sets `enable.idempotence` to true by default, which stops a retry writing a duplicate copy into the log. But it is disabled silently if conflicting configurations are set and it was not explicitly enabled, and it requires `acks=all`, retries above zero, and at most five in flight requests per connection.

Kafka 4.3, Producer Configs

Producer idempotence is not end to end exactly once processing. It means the producer does not duplicate on retry. What your consumer does with the message is a separate problem.

Kafka 4.3, Producer Configs

A Kafka consumer reads uncommitted transactional messages by default. Aborted transaction messages are visible unless the consumer asks for `read_committed`.

Kafka 4.3, Design: Message Delivery Semantics

06 Why not a distributed transaction

The textbook answer to "make the database write and the broker publish atomic" is a distributed transaction. XA is the X/Open standard for one; JTA is the Java API over it. It is a fair thing to raise, and knowing why it is not used is worth more than knowing it exists.

The reason is not that two phase commit is wrong. It is that it takes two willing participants, and the thing on the other side usually cannot play.

Kafka documents two phase commit as the classic way to coordinate a consumer position with its output, and explains that it stores the offset with the output instead, because many of the output systems a consumer writes to will not support a two phase commit.

Kafka 4.3, Design: Message Delivery Semantics

Where XA still genuinely fits is two resources that both implement it, such as a relational database and a JMS broker, under a transaction manager. That is a narrower world than it was, which is why the outbox became the common answer.

First principles, not vendor documentation

What a senior engineer notices

1. What happens when this request is retried
2. What happens when two of these run at the same time
3. What happens if it crashes between the two writes
4. Is the balance the source of truth, or the ledger
5. How do I prove what happened six months from now
6. Which steps must be synchronous, and which are consequences
7. What happens when the notification fails after the money moved
8. What did I deliberately leave out, and can I say why

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Technical claims verified 2026-09-11 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.